other extreme, a copyright message that appears at the bottom of every page of a site is more suited to a text file that is retrieved with the require function.

Somewhere between these two extremes is a break-even point. The reason is that databases provide a tradeoff. They allow you to handle data incomplex ways. They allow you to associate several pieces of information around a common identifier. However, you trade away some performance, as retrieving data is slower than if you opened a file and read the contents.

Many Web sites are nothing more than a handful of pages dressed up in a common graphic theme. A hundred files in a directory are easy to manage. You can name each one to describe their contents and refer to them in a URL such as http://www.mysite.com/index.php?screen= about_us and still get the benefit of systematically generating the layout and navigation. Your PHP script can use the value of the screen variable as the name of a local file, perhaps in a directory named screens. Developers can work on the page contents as they are accustomed, because they know the code is stored in a file named about_us in a directory named screens.

When the content grows to a thousand pages, keeping each in a separate file starts to become unmanageable. A relational database will help you better organize the content. With a site so large it's likely that there will be many versions of the navigation. In a database it is easy to build a table that associates page content with navigation. You can also automate hyperlinks by creating one-way associations between pages. This would cause a link to automatically appear on a page.

The biggest problem with this approach is the lack of good tools for editing the site. Developers are used to fetching files into an editor via FTP. Asking these same people to start using a database shell is most likely out of the question. The cost of teaching SQL to anyone who might work on the site may eliminate any benefit gained when the content is put into the database. So, you are faced with creating your own tools to edit the content. The logical path is to create Web-based tools, since coding a desktop application is a major project in itself, especially if both Windows and Macintosh users are to be accommodated. As you might guess, Web-based site editors are less than ideal. However, with very large sites they become bearable, because the alternative of managing such a large static site is a greater evil, so to speak.

9.9.4 In-Line Debugging

There are times when code produces unexpected results. Examining the code reveals nothing. In this case the best thing to do is some in-line debugging. PHP scripts generate HTML to be interpreted by a browser, and HTML has a comment tag. Therefore, it is a simple matter to write PHP code that reports diagnostic information inside HTML comments